Design and Analysis of Algorithm (DAA), Lab-03

BTech (CS-B and CE), 3^{rd} Semester
August 11, 2026

Instructor: Dr. Ajaya Kumar Dash.

Questions

1. **Binary vs Ternary Search**: In binary search, an n element list is divided into nearly two equal halves, while in ternary search, it is divided into nearly three equal intervals. Then the search will be in one of the intervals. Design and implement a C program to search for an element x in a sorted list of size n using binary and ternary search. Justify and validate that binary search is better than ternary search via your implementation.

2. **Search the Defective Coin**: Imagine you are working as the quality-control engineer for a company that makes coins. The company needs to certify that all the coins must have exactly identical weights. During your inspection, you have observed that one of the workers is watching the mobile phone while giving final shape to a coin. When you confront the worker suddenly, in reflex, he dropped that one coin into a pile of other $(n-1)$ perfectly identical weighted coins. After your confrontation, the worker agreed that he was not attentive and had been shaping the coin for more than the desired duration. However, he could not be sure whether the weight of the coin was less due to excessive shaping or remained perfect, but certain that the weight of the coin must not be more than the required weight. It is your job to find that one possible defective coin that is lighter than the others or possibly none, if, fortunately, the worker has made the perfect coin with exact required weight.

   Being a Computer science graduate, your task is to determine which of the coins is lighter (defective) or report that *none* is lighter. In order to accomplish this task, you have been provided with a balance weighing scale (⚖). Using the balance weighing scale and necessary implementation, provide a divide and conquer algorithm to find the lighter (defective) coin, if present or not, in $\boxed{\log_2 n + c}$ time, for some positive constant c.

3. **Max and Min using D&C Approach**: Using the divide and conquer approach, develop an algorithm to find the maximum and minimum elements in an array of size n so that the number of comparisons will be bounded by $\frac{3n}{2}$. Implement your algorithm in C to validate the result.

4. **Matrix Multiplication using D&C Approach**: Write a C program to multiply two square matrices of size $n \times n$ using Strassen's method.

5. **Multiply special-pattern square matrices using D&C approach**: Two $n \times n$ matrices are provided to you, where $n = 2^k$ for some natural number k. Each matrix has the recursive structure described as: when divided into four equal-sized blocks, two diagonal blocks are

identical, and two off-diagonal blocks are identical, i.e. the structure of the matrices would be:

$$M = \begin{pmatrix} M_1 & M_2 \\ M_2 & M_1 \end{pmatrix}.$$

Each block has a recursive structure that goes down to single integer elements. Give a divide-and-conquer approach-based $O(n^2)$ algorithm for multiplying two such matrices and validate the complexity of your algorithm.

6. **Use of loop invariants in sorting**: Generally, loop invariants are used to prove an algorithm's correctness. To validate the proof, one must show three things about a loop invariant: initialisation, maintenance, and termination. Consider sorting on n numbers stored in an array $A[1 \cdots n]$ by first finding the smallest element of $A[1 \cdots n]$ and exchanging it with the element in $A[1]$. Then, find the smallest element of $A[2 \cdots n]$, and exchange it with $A[2]$. Then, find the smallest element of $A[3 \cdots n]$, and exchange it with $A[3]$. Continue in this manner for the first $(n-1)$ elements of A. Write a pseudocode for this algorithm. What loop invariant does this algorithm maintain? Why does it need to run for only the first $(n-1)$ elements, rather than for all n elements? Give the worst-case running time of the above sorting algorithm in Θ-notation. Is the best-case running time any better?

   Finally, implement your algorithm in C to validate your claim.
